

|

GUÍA 1:

Registro de antecedentes de proyecto

Sigla	Asignatura	Experiencia de Aprendizaje
TPY 1101	Taller Aplicado de Programación	Determinación y planificación de proyecto
Tiempo	Modalidad de Trabajo	Indicadores de logro
3 semanas	Equipos de 3 integrantes	IL 1.1 , 1.2, 1.3, 1.4 y 1.5

Antecedentes generales

Esta guía tiene como propósito registrar todos los antecedentes del proyecto a realizar durante el presente semestre.

Requerimientos para esta actividad

Para el desarrollo de esta actividad los/las estudiantes deberán formar equipos de 3 integrantes, de acuerdo con lo determinado por el/la docente a cargo de la asignatura.

Actividad

Esta actividad tiene como propósito recopilar y organizar información esencial para el proyecto a realizar.

Principalmente, se tendrá que completar los datos que permitan identificar los roles de los integrantes del equipo, la problemática a solucionar, la metodología a implementar, las tecnologías y/o herramientas que utilizaran para el desarrollo de la solución propuesta por el proyecto, las evidencias a construir que respaldaran el trabajo a realizar, la planificación de las tareas a realizar por cada integrante del equipo de trabajo, entre otros antecedentes que apoyen el desarrollo y evidencie los conocimientos adquiridos durante el periodo de formación.

1. Antecedentes Personales

Ingresar la información solicitada de cada integrante del grupo de trabajo

	Integrante 1	Integrante 2	Integrante 3	Integrante 4
Rut	20.455.290-8	20.045.213-5	26.648.362-7	21.749.700.0
cc	Javiera Alexandra	Israel Esteban	Samuel Alexander	Genara
Apellidos	Ramírez Contreras	González Pastén	Bueno Montilla	Alarcon
Roles	Lider/BBDD	Frontend/BBDD	Backend/BBDD	QA/DEVOPS

2. Descripción Proyecto

En la descripción debes señalar brevemente el nombre, descripción, objetivos y alcance del proyecto

Tema	Plataforma Inteligente de Cotización y Comparación de Materiales de Construcción.
Descripción	Permite realizar búsquedas comparativas, gestionar listas de materiales (presupuesto), guardarlos y optimizar la toma de decisiones financieras.
Problemática detectada	Contratistas y particulares deben realizar comparaciones manuales, generando pérdida de tiempo.
Solución esperada	Un ecosistema digital compuesto por un motor de extracción automatizada de datos (Web Scraping), una API robusta de gestión de catálogos y un cotizador dinámico que permita generar presupuestos exportables y derivación directa a los canales de venta oficiales de los retailers.
Objetivos generales	Digitalizar y optimizar el proceso de cotización de materiales de construcción mediante una plataforma centralizada que reduzca los tiempos de búsqueda y mejore la precisión presupuestaria de los proyectos de edificación y remodelación.
Objetivos específicos	Desarrollar una arquitectura de microservicios que integre la extracción automatizada de datos mediante Web Scraping , la gestión de un catálogo normalizado en PostgreSQL con soporte de caché en Redis , y una interfaz en Next.js para la generación de presupuestos técnicos con conversión dinámica a UF .
Alcance del proyecto	El proyecto comprende el desarrollo de una aplicación web que integra los catálogos de al menos tres grandes retailers chilenos. Incluye la búsqueda inteligente, la creación de listas de materiales por proyecto, el cálculo de costos referenciales de flete según ubicación y la exportación de cotizaciones en formatos profesionales (PDF/Excel). El sistema actuará como un agregador y comparador, delegando la transacción final al sitio del proveedor mediante enlaces profundos.

3. Contexto del proyecto

Especificar los antecedentes y motivación para el desarrollo del proyecto, publico objetivo, estado del arte, origen de la problemática y descripción del problema resolver.

La industria de la construcción en Chile enfrenta un escenario de mucha volatilidad económica. La motivación de **ConstructoCompare** nace de la necesidad de digitalizar un proceso que hasta hoy muchos constructores y maestros siguen haciendo de forma arcaica: saltar de pestaña en pestaña entre las webs de Sodimac, Easy e Imperial para comparar precios manualmente y anotarlos en un excel o peor herramientas de texto.

Como estudiantes de ingeniería, sabemos que la ineficiencia es el enemigo. La motivación principal viene a través de la experiencia de varios miembros en el equipo de desarrollo donde han visto y ejecutado esos procesos manuales e ineficientes que se pueden usar para enfocarse en procesos que le generen más valor a quienes realmente les importa los clientes. Nuestro objetivo es democratizar el acceso a la información de mercado en tiempo real, utilizando un stack de alto rendimiento (FastAPI, Next.js) para que el usuario no pierda tiempo ni plata y las pymes puedan hasta competir en igualdad de condiciones con empresas más grandes que tienen los recursos para tener un departamento exclusivo para esas tareas.

Nuestro público objetivo están divididos en 3 actores:

- **Empresas Constructoras (Pymes):** Que necesitan optimizar presupuestos para licitaciones.
- **Maestros y Contratistas:** Profesionales que compran materiales a diario y operan principalmente desde dispositivos móviles en la obra.
- **Autoconstructores:** Personas naturales que buscan el mayor ahorro posible en sus proyectos personales.

El problema central es la **opacidad de precios y la falta de resiliencia en los presupuestos**. Los usuarios no tienen una herramienta que centralice la data de los grandes retailers chilenos de forma dinámica. **ConstructoCompare** resuelve esto mediante un "ID Maestro" que normaliza productos de distintas fuentes, permitiendo generar la cotización más barata para una lista completa de materiales, asegurando que la información sea transparente incluso cuando los sistemas de origen fallan.

4. Metodología

Describir la metodología a utilizar para el desarrollo del proyecto y justificar su uso.

Descripción Metodología

Para el desarrollo de **ConstructoCompare Pro**, utilizaremos la metodología ágil **Scrum**, debido a su capacidad de adaptación ante cambios en los requerimientos (especialmente críticos cuando dependemos de estructuras externas de terceros).

Justificación: La volatilidad de los selectores CSS en las webs de los retailers exige un desarrollo iterativo. Scrum nos permite realizar entregas funcionales (Sprints) de módulos específicos (ej. el motor de scraping de una sola tienda) antes de integrar el sistema completo.

Las etapas que tendrá el proyecto son:

- **Ingestión de Datos (Data Ingestion)** Se emplea **Python con Playwright** para realizar web scraping asíncrono. El método consiste en simular sesiones de usuario para evitar bloqueos y capturar la carga dinámica de JavaScript. Los datos se extraerán en crudo (Raw Data) para su posterior procesamiento.



- **Procesamiento y Normalización (Data Processing)** Aquí entra en juego el **ID Maestro**. Utilizaremos lógica de limpieza de strings y algoritmos de coincidencia (Fuzzy Matching) para normalizar nombres de productos. El procesamiento se realizará en el backend mediante **FastAPI**, validando los esquemas con **Pydantic** para asegurar la integridad de la información.
- **Persistencia y Caché** Los datos históricos se almacenarán en **PostgreSQL**, mientras que los precios en tiempo real y resultados de búsqueda frecuentes se gestionarán en **Redis** para garantizar tiempos de respuesta eficientes, emulando la eficiencia de una plataforma de grado financiero.
- **Visualización y Despliegue** El frontend en **Next.js** consumirá las APIs del backend. El despliegue se automatizará mediante **Vercel**.

- **Javiera Alexandra Ramírez Contreras** asume el rol de **Project Lead, QA y Administradora de BBDD**. Debido a su capacidad organizativa, lidera la gestión del proyecto bajo el marco de trabajo Scrum, encargándose de la planificación de Sprints y el cumplimiento de los hitos establecidos. Como administradora de la base de datos, asegura la correcta implementación del esquema lógico y la integridad de las reglas de negocio en la persistencia de datos.
- **Israel Esteban González Pastén** se desempeña como **Frontend Developer y colaborador en BBDD**. Su responsabilidad central es el desarrollo de la interfaz de usuario utilizando Next.js, garantizando una experiencia fluida y un diseño responsivo para dispositivos móviles. Se encarga de la lógica de consumo de APIs asíncronas y de la gestión de estados en el cliente, además de colaborar en la implementación de consultas eficientes en la base de datos para asegurar que la visualización de precios y comparativas sea instantánea.
- **Samuel Alexander Bueno Montilla** actúa como **Backend Developer y Diseñador de BBDD**. Como responsable técnico de la arquitectura de datos, es el encargado de la lógica de negocio mediante FastAPI. Su labor se enfoca en la ingeniería de procesos para la extracción de datos (Scraping) con Playwright y la normalización de productos mediante el "ID Maestro". Garantiza que el flujo de información desde los retailers hasta la base de datos sea robusto, automatizado y optimizado para el rendimiento del sistema.
- **Genara Alarcón: QA & DevOps Engineer** se incorpora para garantizar la fiabilidad del software y la continuidad operativa del sistema. En su rol de **QA (Quality Assurance)**, es responsable de la creación y ejecución de planes de prueba, validando la integridad de los datos mediante esquemas de **Pydantic** y supervisando que cada incremento de software cumpla con el *Definition of Done*. Como **DevOps**, gestiona el despliegue continuo (CI/CD) en la nube y la optimización de recursos en el servidor, asegurando que la plataforma mantenga altos estándares de disponibilidad y rendimiento bajo carga.

5. Descripción técnica de la solución

Describir los detalles técnicos del producto a desarrollar



Especificaciones técnicas de la solución a desarrollar

Infraestructura a utilizar: Se implementa una arquitectura de **entorno híbrido de desarrollo (On-premise/Cloud)**. El núcleo de procesamiento y la capa de ingestión (scraping) operan en un nodo local orquestado mediante **Docker**, permitiendo una portabilidad inmediata a servicios como AWS o Azure en fases de escalado. La capa de presentación (Frontend) se despliega en **Vercel** para garantizar acceso público y alta disponibilidad del cliente.

Se utiliza un modelo de capas: **Client Layer** (interfaz de usuario), **Service Layer** (lógica de negocio asíncrona) y **Data Ingestion Layer** (extracción resiliente). La orquestación de datos se maneja mediante **Cron Jobs** estratégicos a las 03:00 AM y 12:00 PM

Python para el backend y la ingeniería de datos. **JavaScript/TypeScript** para la lógica del lado del cliente y renderizado en servidor.

FastAPI para el backend debido a su velocidad y validación nativa con **Pydantic**. **Next.js** para el frontend, priorizando el SEO y SSR. Para la extracción, se usa **Playwright** para emular navegadores y evadir bloqueos de carga dinámica (JS).

Un sistema dual: **PostgreSQL** para la persistencia relacional y almacenamiento de históricos de hasta 90 días. **Redis** como capa de caché en RAM para garantizar respuestas ultra rápidas menores a 200ms

Repositorio:

Github: <https://github.com/Creative-Duoc/ContructoCompare>

drive:

https://drive.google.com/drive/folders/1ibyCxT5ajdH2HDXXFtzLWfeWFGqXTiEk?usp=drive_link

Página Web: <https://contructo-compare.vercel.app/app>

Conexión vía API con **mindicador.cl** para conversión automática en UF.

6. Plan de Trabajo

Define la planificación de tu Proyecto de a las evidencias a realizar durante el semestre.

Nr o.	Nombre de Actividades/Tareas	Descripción Actividades/Tareas	Recursos	Duración de la actividad	Responsable	Observaciones
1. -	Levantamiento y arquitectura	Diseño de diagramas de microservicios, modelado inicial de la BBDD, creación de mockup de la pagina y definición de contratos API	Draw.io, Lucidchart, Google Docs.	2 semana	Samuel, Javiera e Israel	Hito crítico para inicio de dev.
2. -	Ingeniería de Ingesta (Scraper)	Desarrollo del motor Playwright para Sodimac, Easy e Imperial con lógica de bypass.	Python, Playwright, GitHub.	2 semanas	Samuel, Javiera e Israel	Requiere pruebas de rotación de Headers.
3. -	Modelado y Persistencia	Configuración de PostgreSQL y lógica de ID Maestro para normalización de SKUs.	PostgreSQL, DBeaver, SQLAlchemy.	2 semanas	Javiera, Samuel	Clave para evitar duplicidad de productos .
4. -	Capa de Negocio y UF	API en FastAPI con integración a Mindicador.cl y lógica de cálculo de presupuestos.	FastAPI, Python, Postman.	2 semanas	Israel, Samuel	Implementación de Fallback Logic.
5. -	Frontend UX/UI	Desarrollo de interfaz en Next.js, buscador dinámico y exportación a PDF.	Next.js, Tailwind CSS, Vercel.	3 semanas	Israel	Optimización para carga móvil en obra.



6.	QA y Optimización Redis	Pruebas de estrés, implementación de caché en Redis y pulido de errores.	Redis, Pytest, Docker	1 semana	Genara	Meta: Respuesta < 200ms.
----	-------------------------	--	-----------------------	----------	--------	--------------------------

7. Evidencias

Declarar las evidencias a diseñar para el proyecto. Estas evidencias deben ser acordadas con el docente, dependiendo de la dimensión del proyecto. Todas las evidencias deben estar en formato .pdf, .xls, .xlsx, .doc o .docx, según corresponda y deben ser legibles para el docente.

Nr o.	Nombre de evidencia	Descripción	Link para acceder (repositorio)
1	Documentación Técnica Inicial	Contiene Mapa Mental, Impact Mapping, MER y Diagrama de Arquitectura.	
2	Product Backlog & User Stories	Listado priorizado de Épicas e Historias de Usuario con criterios de aceptación.	
3	Script de BBDD y Migraciones	Archivos .sql con la estructura normalizada y datos semilla para pruebas.	
4	Prototipo Funcional (MVP)	Código fuente de los microservicios de scraping y API base operativa.	
5	Informe de Resiliencia y QA	Reporte de pruebas de fallos de scraping y tiempos de respuesta con Redis.	
6	Presentación Final	PPT de defensa	

8. Planificación semestral

Describir todas las tareas a realizar durante el semestre, considerando los hitos de entregas en cada una de las fases.

Actividad/semana	Fase 1				Fase 2								Fase 3				Presentación Final	
	S 1	S 2	S 3	S 4	S 5	S 6	S 7	S 8	S 9	S 10	S 11	S 12	S 13	S 14	S 15	S 16	S 17	S 18
Fase 1: Análisis y Diseño	x	x																
Fase 2: Dev Ingesta & BBDD			x	x	x													
Fase 3: Dev API & Lógica					x	x	x	x										
Fase 3: Dev Front & Integración								x	x	x								
QA, Redis & Fallback											x	x						
Ensayo presentación													x	x	x	x		

9. Otros

Detallar antecedentes adicionales que enriquezcan el presente proyecto o proporcionar evidencia adicional solicitada por el docente para respaldar el presente proyecto.

Se adjunta la documentación que son el documento Planificación ágil y Gestión del producto, y los Informe de Gestión y Cierre de la totalidad de los 5 sprint que respaldan la realización del proyecto.